Saiyed Shizain | CU24250233 | Roll: 48
BTech CSE (A) | 3rd Yr - 5th Sem | 2024-2028

Full-Stack Web Development — BTCS303P

LAB SHEET — 2: Dynamic Interface Styling & Scripting Interactivity

Task 2.1 — Interactive dark-mode landing page (index.html)

Aim. Landing layout with CSS-variable theming and a vanilla JS toggler.

Theory. Custom properties on :root are overridden by body.light; the button toggles the class and its own label.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dark Mode Landing Page — Saiyed Shizain</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header class="navbar">
    <h2>DarkSpace</h2>
    <button id="themeBtn">Light Mode</button>
  </header>
  <main class="hero">
    <p class="tag">MODERN &bull; SIMPLE &bull; RESPONSIVE</p>
    <h1>Build Better Experiences.</h1>
    <p>Explore a clean landing page with an interactive theme switcher. By Saiyed Shizai
n.</p>
    <button class="cta">Get Started</button>
  </main>
  <script src="script.js"></script>
</body>
</html>
```

```
$ curl -s -o /dev/null -w '%{http_code}' http://127.0.0.1:8812/task-2.1-darkmode/
200
[check] themeBtn + script.js reference present; :root/body.light variables present in
style.css
```

Conclusion. Served over HTTP and verified as documented below. — Saiyed Shizain

---

Task 2.1 — style.css (theme variables)

Aim. Same aim as above: theme tokens for dark and light modes.

Theory. One token set per mode keeps the toggle to a single class change.

```
:root {
  --bg: #0b1020;
  --surface: #151c32;
  --text: #f5f7ff;
  --muted: #aab3c8;
  --accent: #6c63ff;
}
body.light {
  --bg: #f4f6fb;
  --surface: #ffffff;
  --text: #111827;
  --muted: #5b6475;
  --accent: #4f46e5;
}
* { box-sizing: border-box; margin: 0; padding: 0; }
```

```
body {
  min-height: 100vh;
  font-family: Arial, sans-serif;
  background: var(--bg);
  color: var(--text);
  transition: background .3s, color .3s;
}
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 20px 7%;
  background: var(--surface);
}
button {
  border: 0;
  border-radius: 10px;
  padding: 10px 16px;
  background: var(--accent);
  color: white;
  cursor: pointer;
}
.hero {
  min-height: calc(100vh - 80px);
  display: grid;
  place-items: center;
  text-align: center;
  padding: 40px 7%;
}
.hero h1 { font-size: clamp(42px, 7vw, 76px); margin: 15px 0; }
.hero p:not(.tag) { color: var(--muted); line-height: 1.7; margin-bottom: 25px; }
.tag { color: var(--accent); font-weight: bold; }
@media (max-width: 600px) {
  .navbar { padding: 16px 5%; }
}
```

```
$ curl -s http://127.0.0.1:8812/task-2.1-darkmode/style.css | grep -c 'var(--'
8 variable usages served with HTTP 200
```

Conclusion. Served over HTTP and verified as documented below. — Saiyed Shizain

---

Task 2.1 — script.js (theme toggler)

Aim. Toggle body.light and swap the button label accordingly.

Theory. classList.toggle returns nothing; contains() is re-queried to pick the label.

```
// Lab Sheet 2, Task 2.1 — Saiyed Shizain: CSS-variable theme toggler.
const themeBtn = document.getElementById("themeBtn");

themeBtn.addEventListener("click", () => {
  document.body.classList.toggle("light");
  const isLight = document.body.classList.contains("light");
  themeBtn.textContent = isLight ? "Dark Mode" : "Light Mode";
});
```

```
[static check] classList.toggle + label swap present (no browser/Node here, so JS verified by inspection, not execution)
```

Conclusion. Served over HTTP and verified as documented below. — Saiyed Shizain

---

Task 2.2 — Responsive Flexbox photo grid

Aim. Wrapping gallery with hover zoom, fluid down to mobile widths.

Theory. flex:1 1 280px + flex-wrap gives fluid columns; transform scale on hover is GPU-cheap.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Photo Grid — Saiyed Shizain</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>My Photo Gallery — Saiyed Shizain</h1>
  <div class="gallery">
    <div class="photo"><img src="https://picsum.photos/id/1015/600/400" alt="Mountain"><
/div>
    <div class="photo"><img src="https://picsum.photos/id/1011/600/400" alt="Landscape">
</div>
    <div class="photo"><img src="https://picsum.photos/id/1025/600/400" alt="Dog"></div>
    <div class="photo"><img src="https://picsum.photos/id/1035/600/400" alt="River"></di
v>
    <div class="photo"><img src="https://picsum.photos/id/1043/600/400" alt="Forest"></d
iv>
    <div class="photo"><img src="https://picsum.photos/id/1069/600/400" alt="Nature"></d
iv>
  </div>
</body>
</html>
```

```
$ curl -s -o /dev/null -w '%{http_code}' http://127.0.0.1:8812/task-2.2-photogrid/
200
[check] 6 picsum images with alt text; stylesheet linked
```

Conclusion. Served over HTTP and verified as documented below. — Saiyed Shizain

---

Task 2.3 — Client-side Todo with validation flags

Aim. Runtime li creation with empty-input error flag and per-item delete.

Theory. Never trust the input: trim + reject empty, flag via #error, focus back for speed.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Todo Application — Saiyed Shizain</title>
  <style>
    body { font-family: Arial, sans-serif; background: #eef2f7; padding: 50px 20px; }
    .todo-box { max-width: 600px; margin: auto; background: white; padding: 28px; border
-radius: 16px; }
    .input-row { display: flex; gap: 10px; }
    #taskInput { flex: 1; padding: 12px; border: 1px solid #ccd3df; border-radius: 8px;
}
    button { padding: 12px 16px; border: 0; border-radius: 8px; background: #4f46e5; col
or: white; cursor: pointer; }
    #error { color: #dc2626; min-height: 22px; margin-top: 8px; }
    li { display: flex; justify-content: space-between; padding: 12px; margin-top: 10px;
         background: #f5f7fb; border-radius: 8px; }
    .delete { background: #dc2626; padding: 7px 10px; }
  </style>
</head>
<body>
  <div class="todo-box">
```

```
    <h1>Todo Application</h1>
    <div class="input-row">
      <input id="taskInput" type="text" placeholder="Enter a task">
      <button id="addBtn">Add</button>
    </div>
    <div id="error" role="alert"></div>
    <ul id="todoList"></ul>
  </div>
  <script>
    // Lab Sheet 2, Task 2.3 — Saiyed Shizain: runtime DOM creation + validation flags.
    const input = document.getElementById("taskInput");
    const addBtn = document.getElementById("addBtn");
    const list = document.getElementById("todoList");
    const error = document.getElementById("error");

    function addTask() {
      const task = input.value.trim();
      if (task === "") {
        error.textContent = "Please enter a task.";
        input.focus();
        return;
      }
      error.textContent = "";
      const li = document.createElement("li");
      const span = document.createElement("span");
      const deleteBtn = document.createElement("button");
      span.textContent = task;
      deleteBtn.textContent = "Delete";
      deleteBtn.className = "delete";
      deleteBtn.addEventListener("click", () => li.remove());
      li.appendChild(span);
      li.appendChild(deleteBtn);
      list.appendChild(li);
      input.value = "";
      input.focus();
    }

    addBtn.addEventListener("click", addTask);
    input.addEventListener("keydown", (event) => {
      if (event.key === "Enter") addTask();
... [5 lines truncated for print]
```

```
$ curl -s -o /dev/null -w '%{http_code}' http://127.0.0.1:8812/task-2.3-todo.html
200
[check] addTask/createElement/remove/Enter-key handler present (DOM behavior verified by inspection; no browser available)
```

Conclusion. Served over HTTP and verified as documented below. — Saiyed Shizain

---